

# 两轮车雷达使用手册

|    |    |           |
|----|----|-----------|
|    | 姓名 | 日期        |
| 编制 | 余枫 | 2026-5-19 |
| 评审 |    |           |
| 审批 |    |           |

## 版本控制

| 版本号  | 变更内容                             | 变更时间      | 变更人 |
|------|----------------------------------|-----------|-----|
| V0.1 | 草稿                               | 2026-5-19 | 余枫  |
| V1.0 | 增加 galget serial 相关内容<br>更新雷达板照片 | 2026-6-16 | 余枫  |
| V1.1 | 增加配置参数写入操作方法                     | 2026-6-30 | 余枫  |
| V1.2 | 增加水印<br>增加目录                     | 2026-7-9  | 余枫  |
| V1.3 | 增加 OTA 升级功能                      | 2026-7-17 | 余枫  |
| V1.4 | 增加 ADC 输出功能<br>增加常用指令集           | 2026-7-22 | 余枫  |

## 目录

|                             |    |
|-----------------------------|----|
| 版本控制.....                   | 2  |
| 一、 平台介绍 .....               | 4  |
| 二、 升级流程 .....               | 6  |
| 三、 Gadget serial 通讯说明 ..... | 11 |
| 四、 配置参数使用说明 .....           | 14 |
| 五、 OTA 升级操作 .....           | 18 |
| 1 首次 OTA 下载 .....           | 18 |
| 2 OTA 升级流程 .....            | 21 |
| 六、 ADC 数据采集 .....           | 22 |
| 七、 常用指令集 .....              | 27 |

## 一、平台介绍

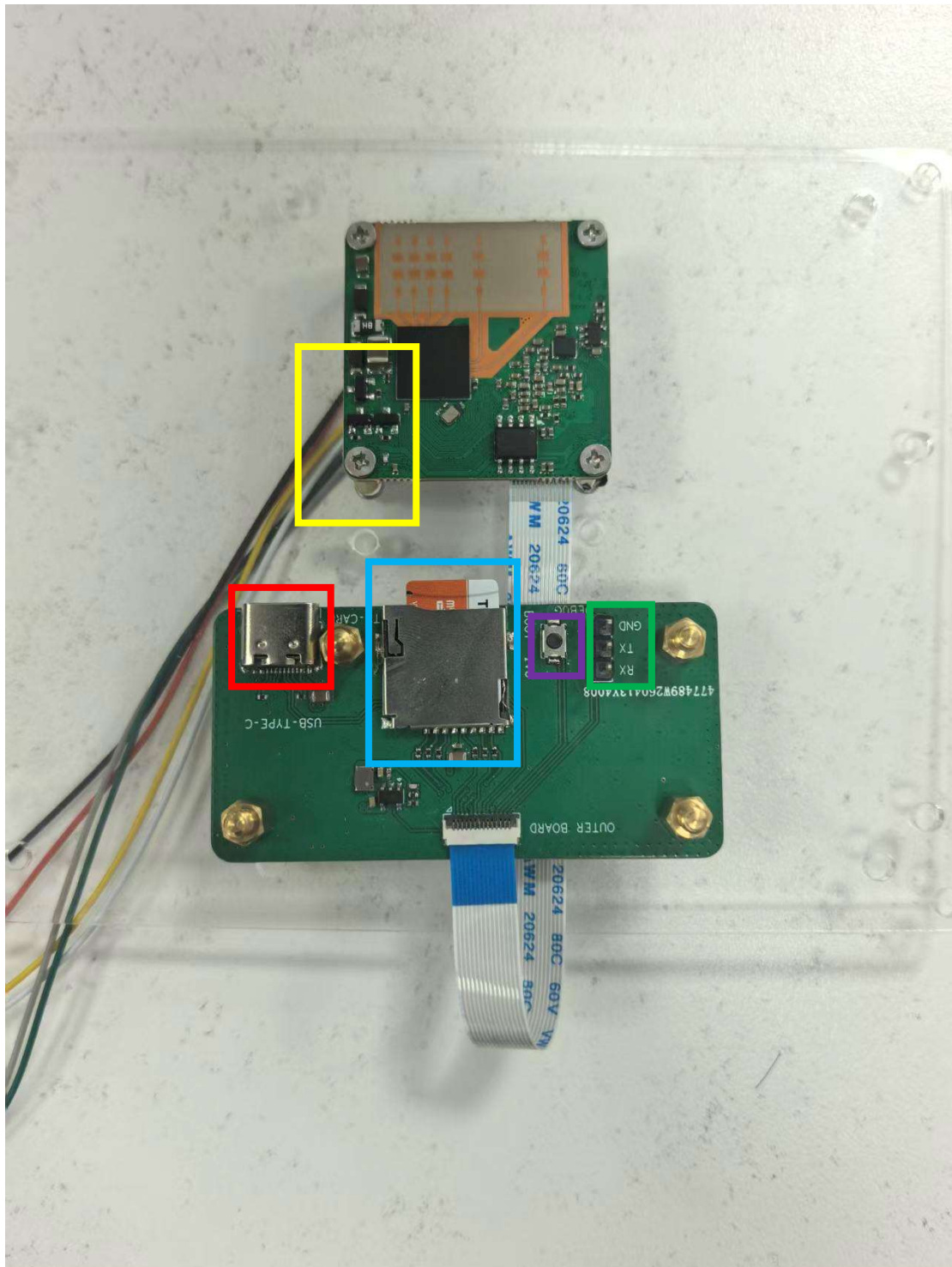


图 1 雷达 Demo 板

1. 黄色框中的线束为轨迹输出端口
2. 绿色框中的 UART 端口为打印信息端口
3. 蓝色框中的 SD 卡用于存储雷达运行时采集的原始数据

4. 红色框是雷达板的供电端口, 固件升级端口和 galget serial 端口
5. 紫色框是雷达板升级前需要按下复位按钮, 然后雷达上电。

SenardMicro

## 二、升级流程

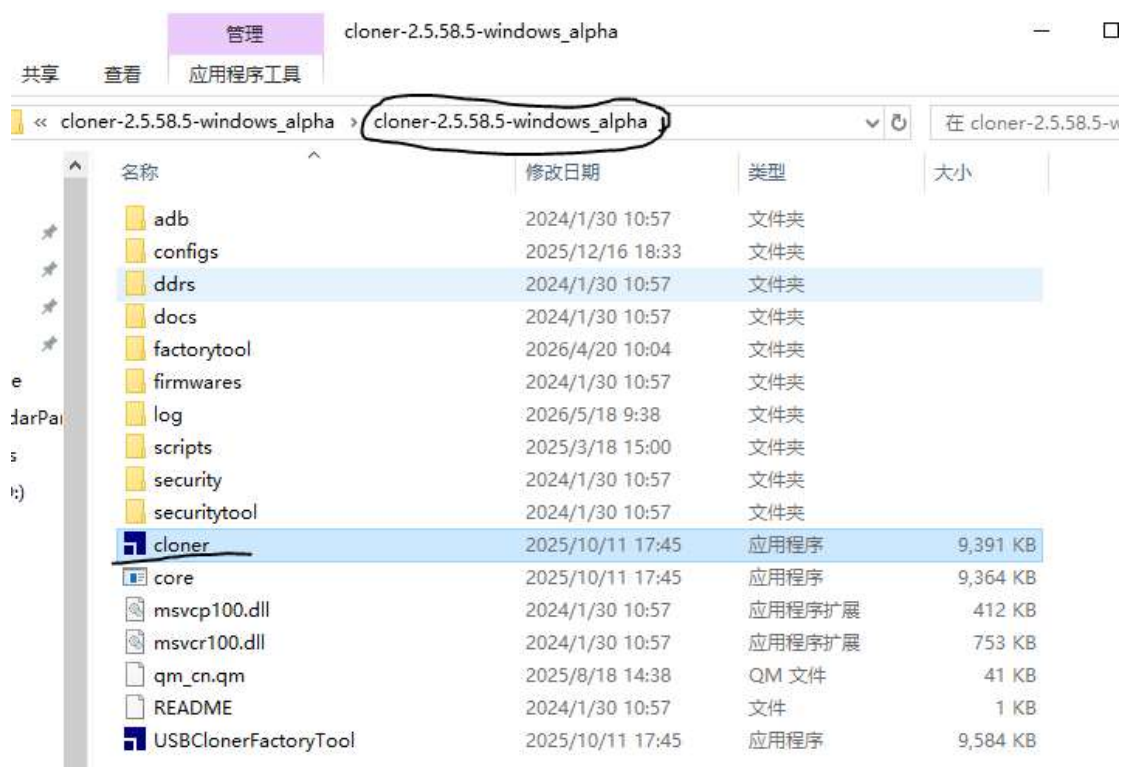


图 2 升级上位机位置

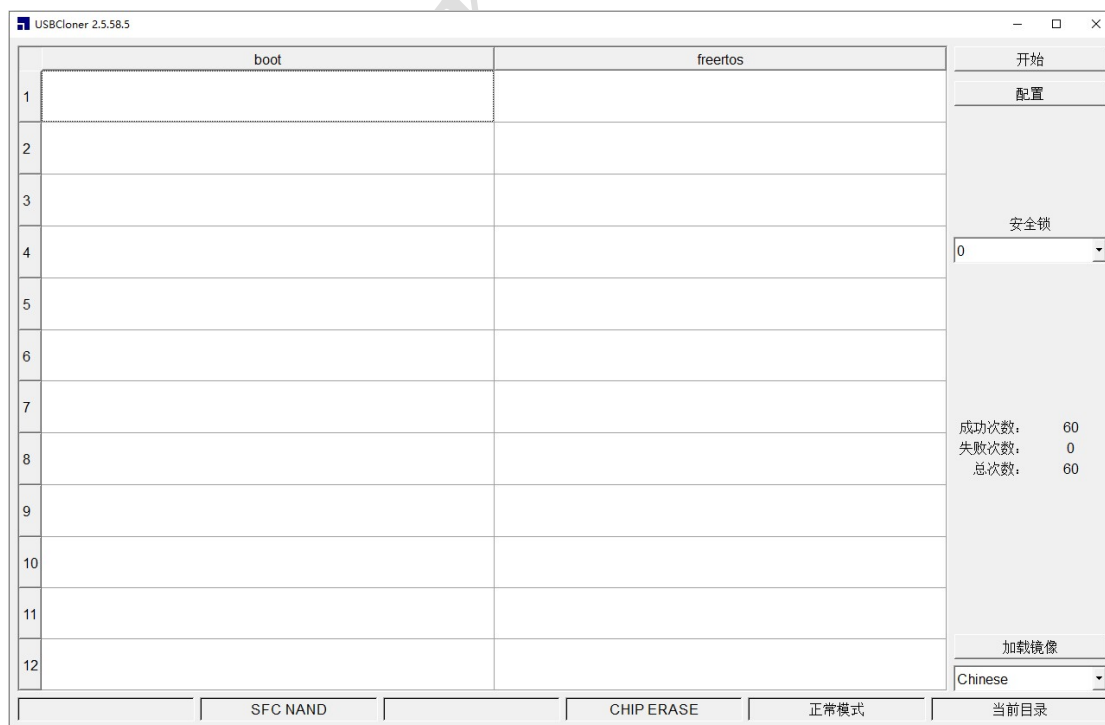


图 3 升级上位机

打开升级上位机后，点击配置：



图 4 选择 X2000 主芯片平台

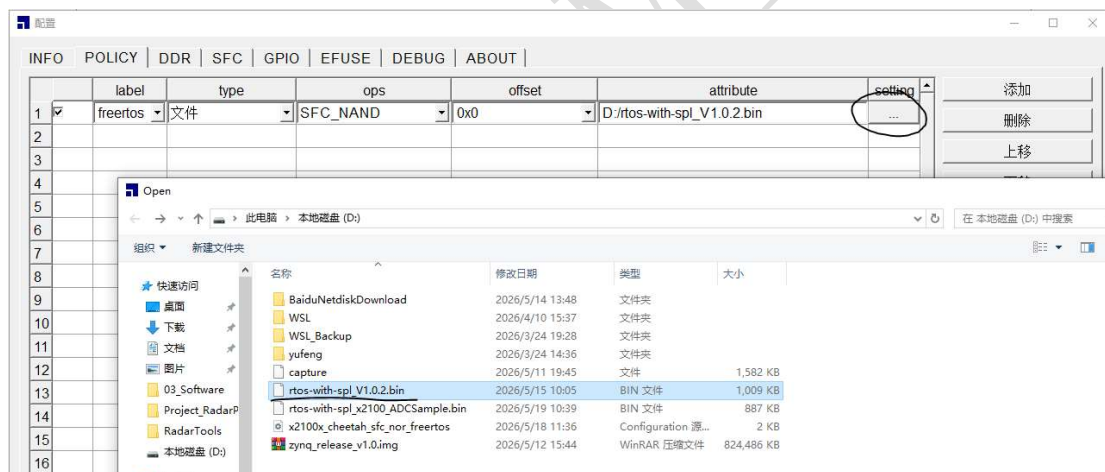


图 5 选择升级文件

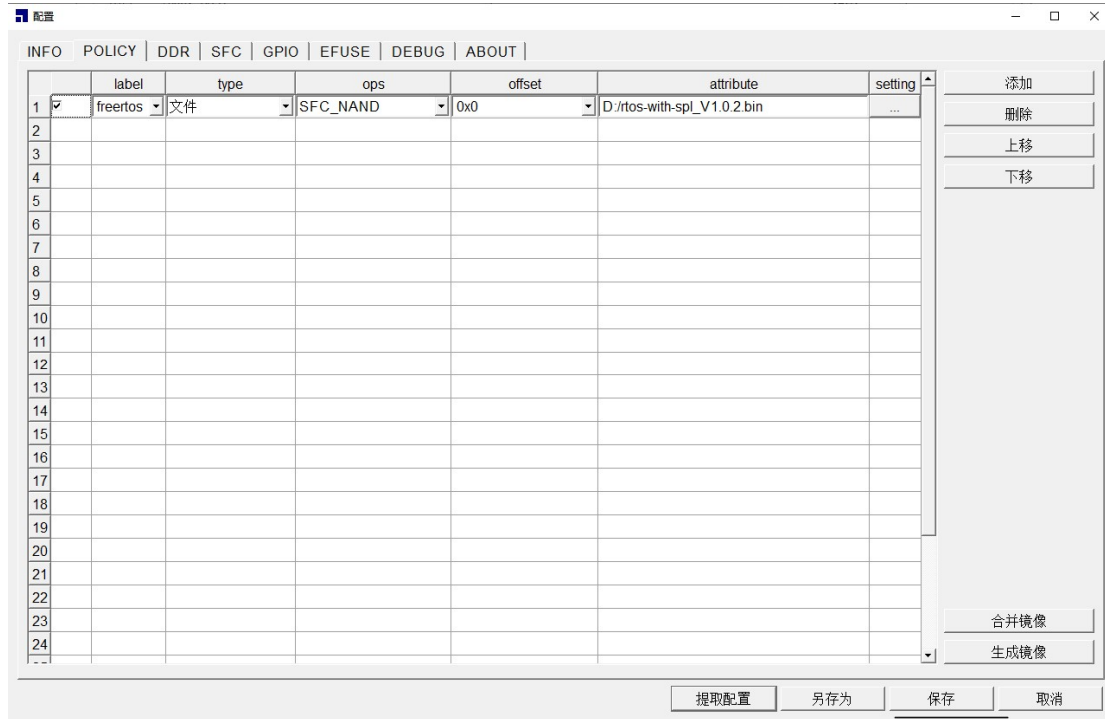


图 6 保存配置信息

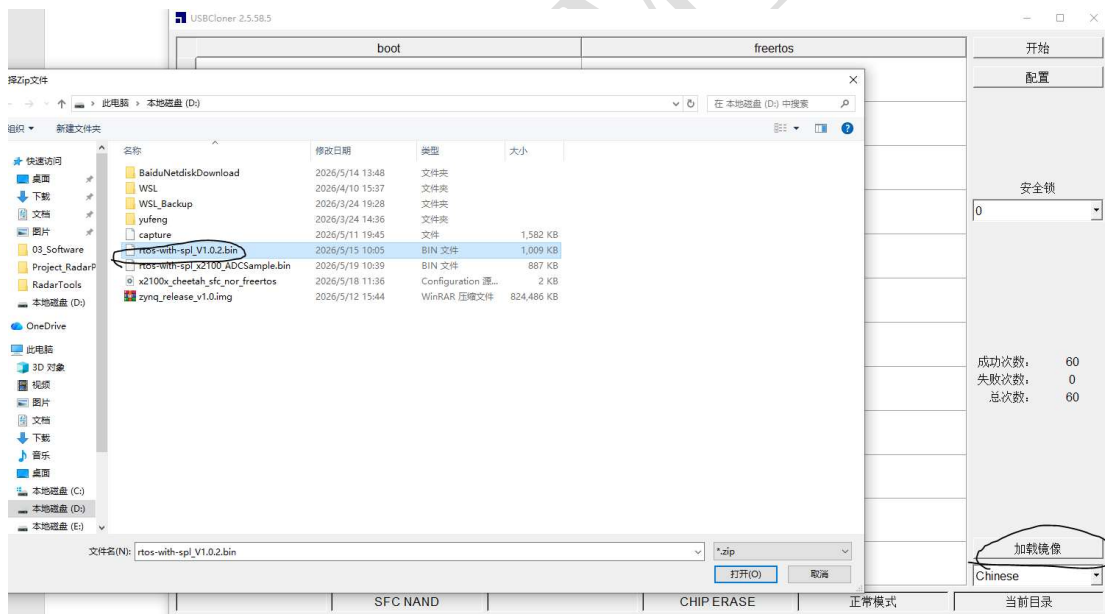


图 7 回到主页面，再次加载镜像

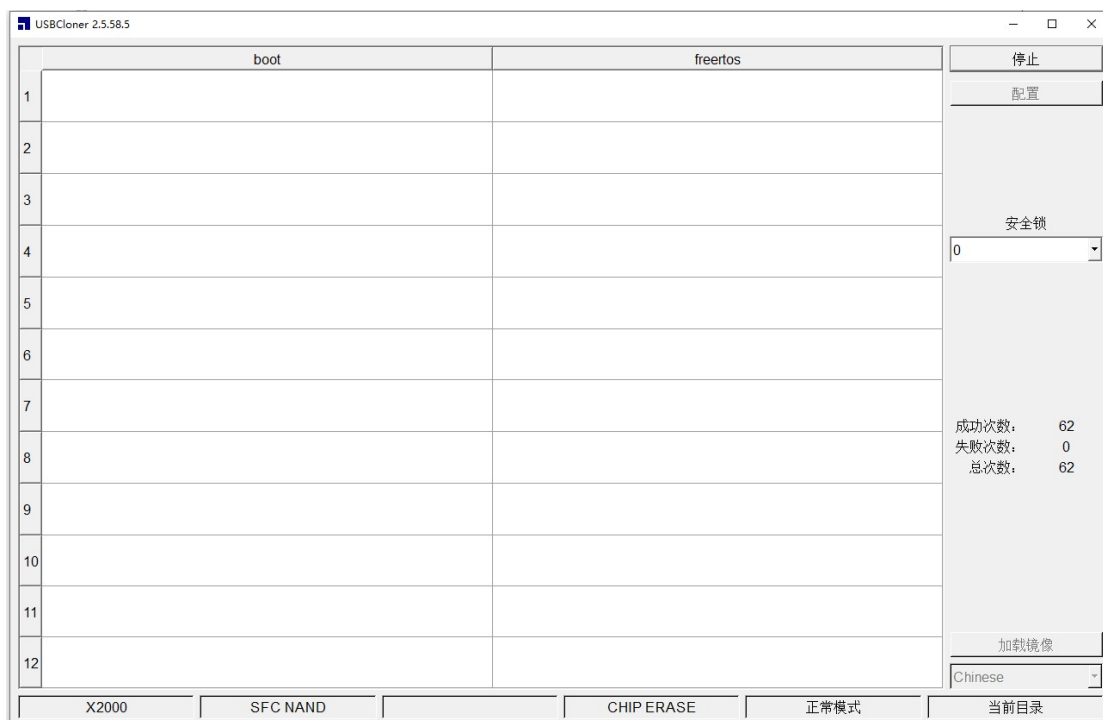
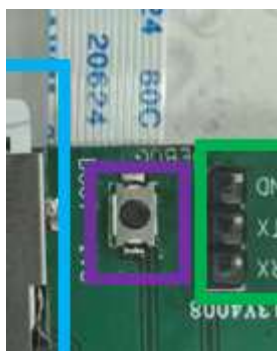


图 8 点击开始后，开始按钮变成停止  
当上位机变成上述界面后，用户按住雷达板上的升级按钮



然后再对雷达的 Type-C 端口上电



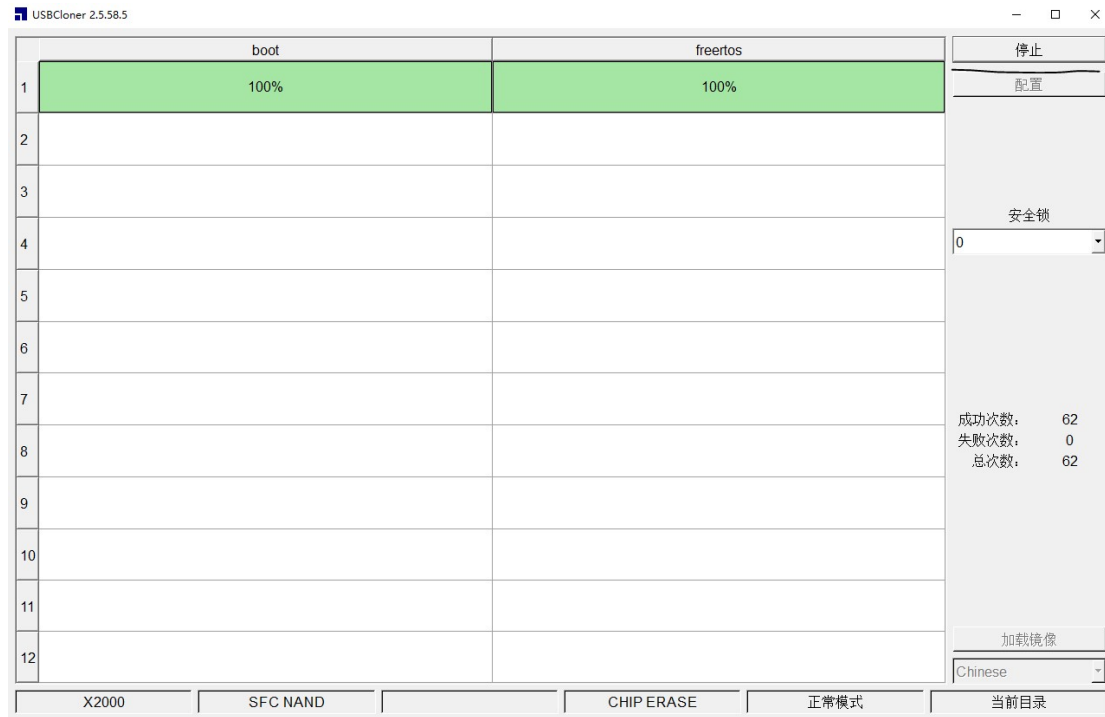


图 9 上电后，雷达自动完成升级，如上图所示

再次断电后上电，雷达开始正常工作，打印信息如下图所示并开始正常运行。默认雷达上电后运行 600 个 Frame，然后停止运行。

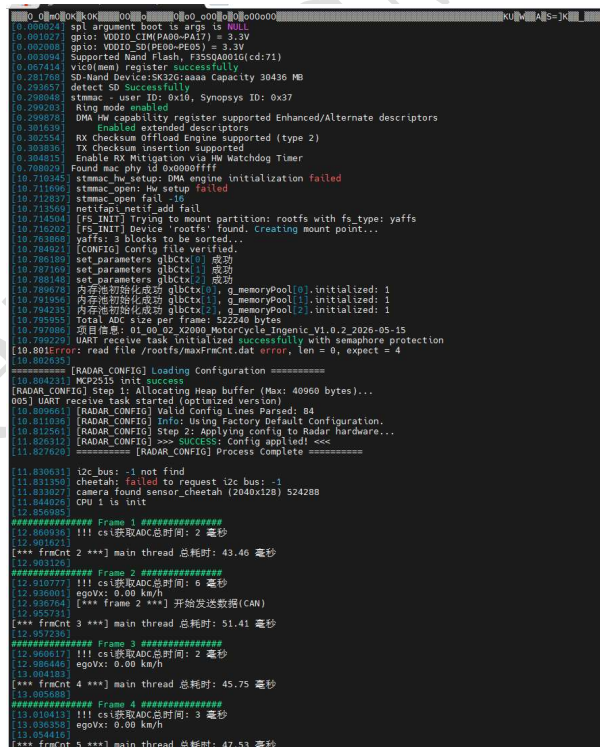


图 10 运行时的打印信息

轨迹信息请参考上位机文档《MotorCycle Tools 使用说明[2.2.9]》

## 三、Gadget serial 通讯说明

### 1. 修改代码

为了增强通讯的稳定性和可靠性，后续会采用 gadget serial 的通讯方式采集数据。

- 通过UART进行数据回放10000帧，两组UART，如下，存在大量的不一致，可以确定UART的数据稳定性较差。

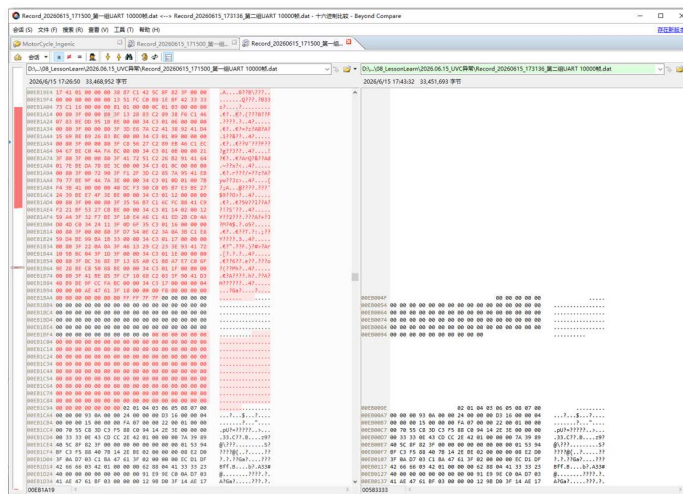
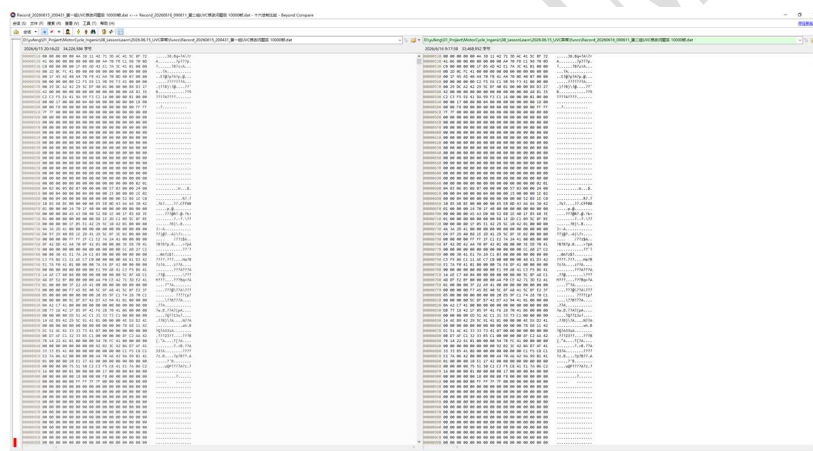


图 11 普通 UART 输出路试数据回放结果，同样的数据回放结果存在差异



- 可以看到除了最后一部分数据，因为左边的测试组采集到10323帧左右，多了一部分数据。其他部分都是字节级一致的。

图 12 使用 gadget serial 的回放结果是字节级一致的  
因此开发了 gadget serial 的通讯模块，代码中的开启方式如下图所示：

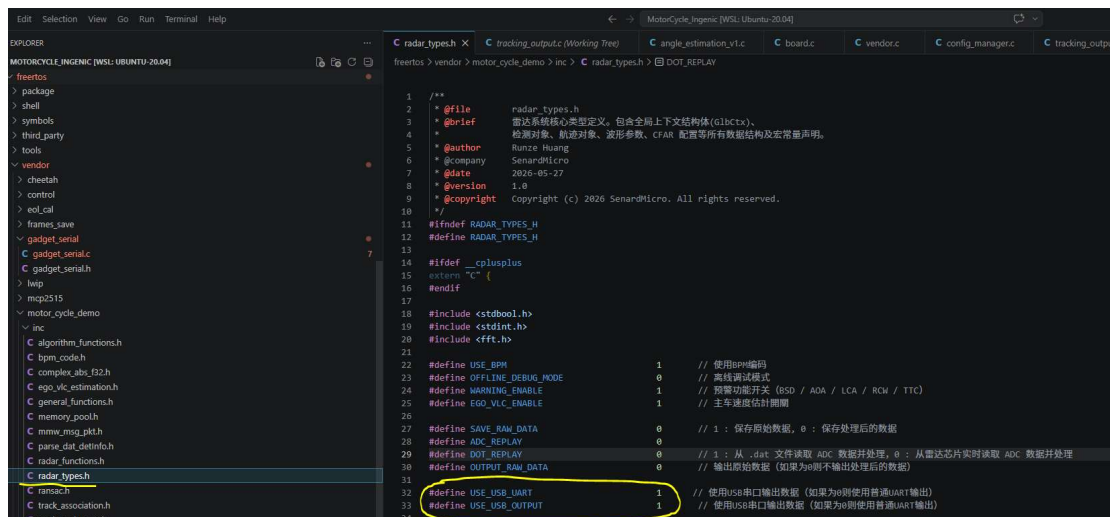


图 13 在代码中将宏设置为 1

将代码中的宏 USE\_USB\_UART 和 USE\_USB\_OUTPUT 设置为 1，开启 galget serial 输入指令和输出信息功能。

## 2. 上位机操作:

在雷达断电后，galget serial 是从设备列表中丢失的，所以打开上位机后，一开始找不到设备。



图 14 设备管理器状态

雷达上电并完成初始化以后，系统能够检索到设备

## 3. 然后按照常规操作，选择 COM17 端口并连接，就可以接收到点云和轨迹信息了

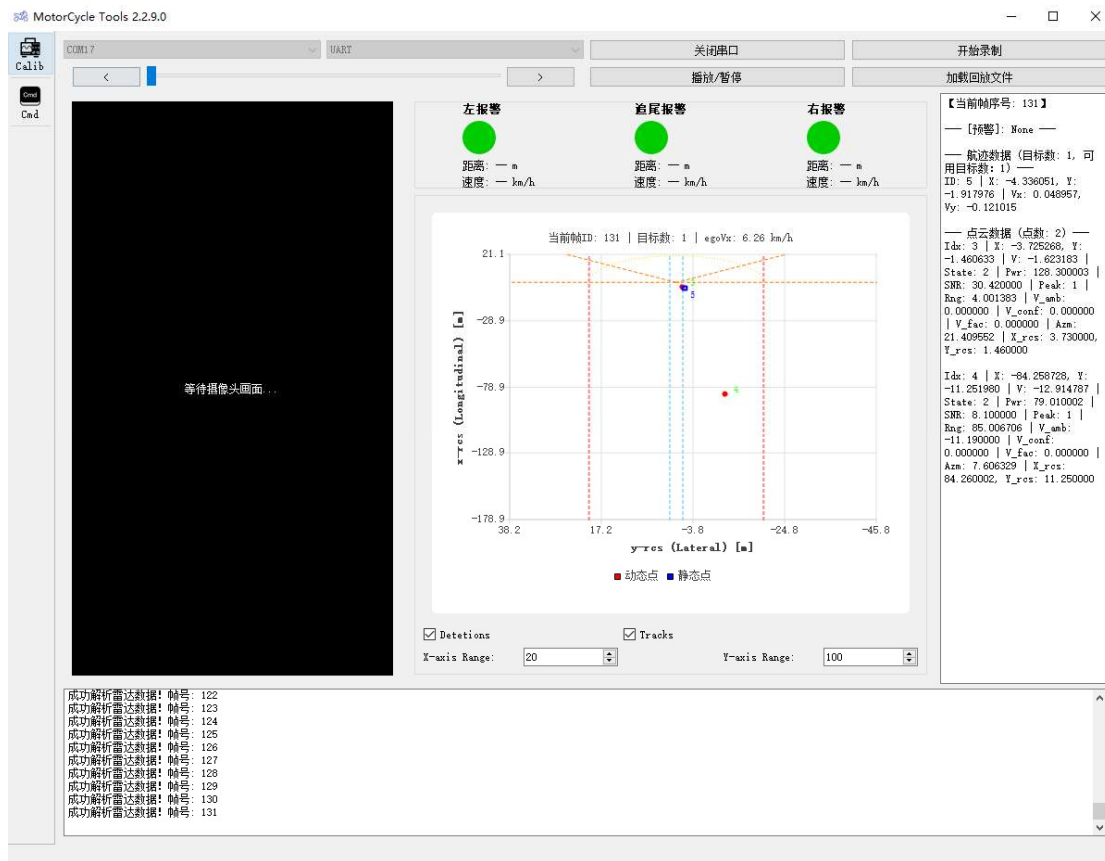


图 15 上位机显示

#### 4. 雷达控制指令发送:

由于目前的上位机不能发送指令, 所以需要使用串口工具发送指令。先断开显示上位机, 然后连接串口工具, 并下发指令。可以看到雷达执行了对应的指令。



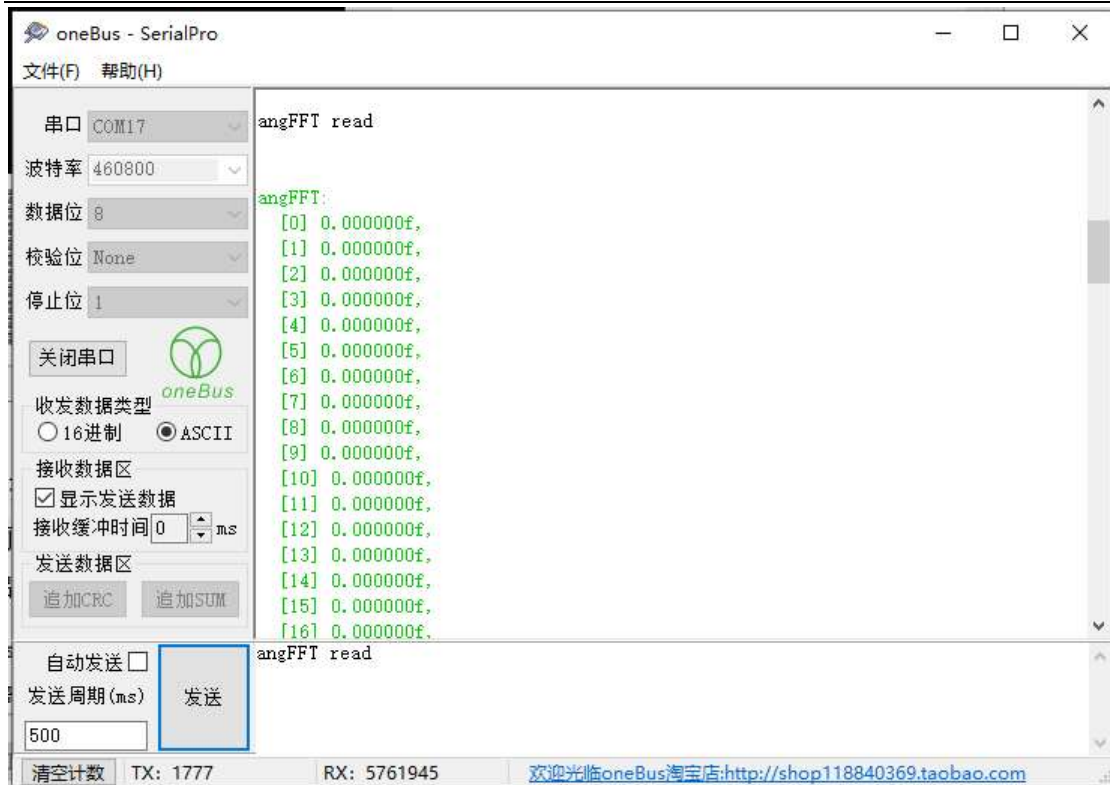


图 16 串口指令下发

## 四、配置参数使用说明

1. 修改后的配置文件存放在地址 0x190000~0x19FFFF 这个区块

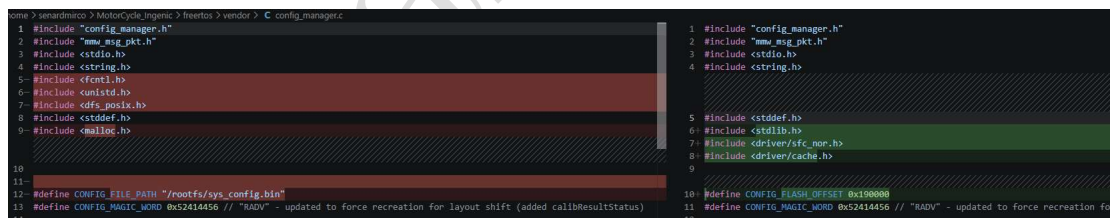


图 17 配置参数地址修改

2. 下载工具的配置文件修改:

为保证下载时不会对配置参数的区块进行擦除, 下载工具的配置文件要进行如下修改:

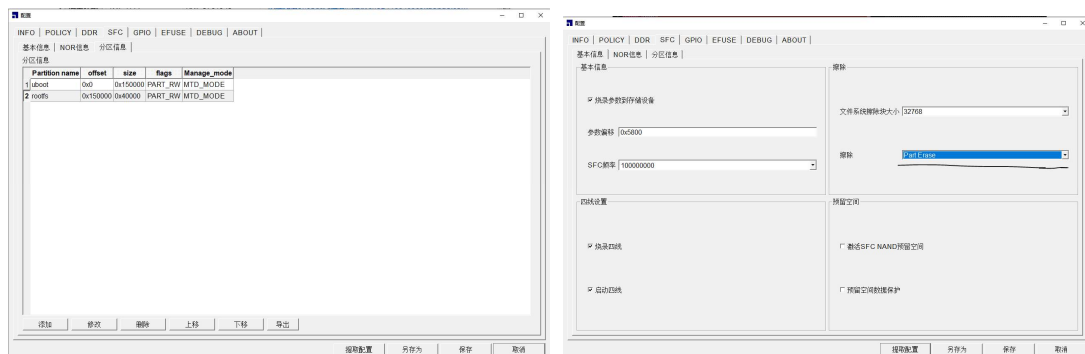


图 18 配置工具的下载文件配置

### 3. 打开配置文件下载工具

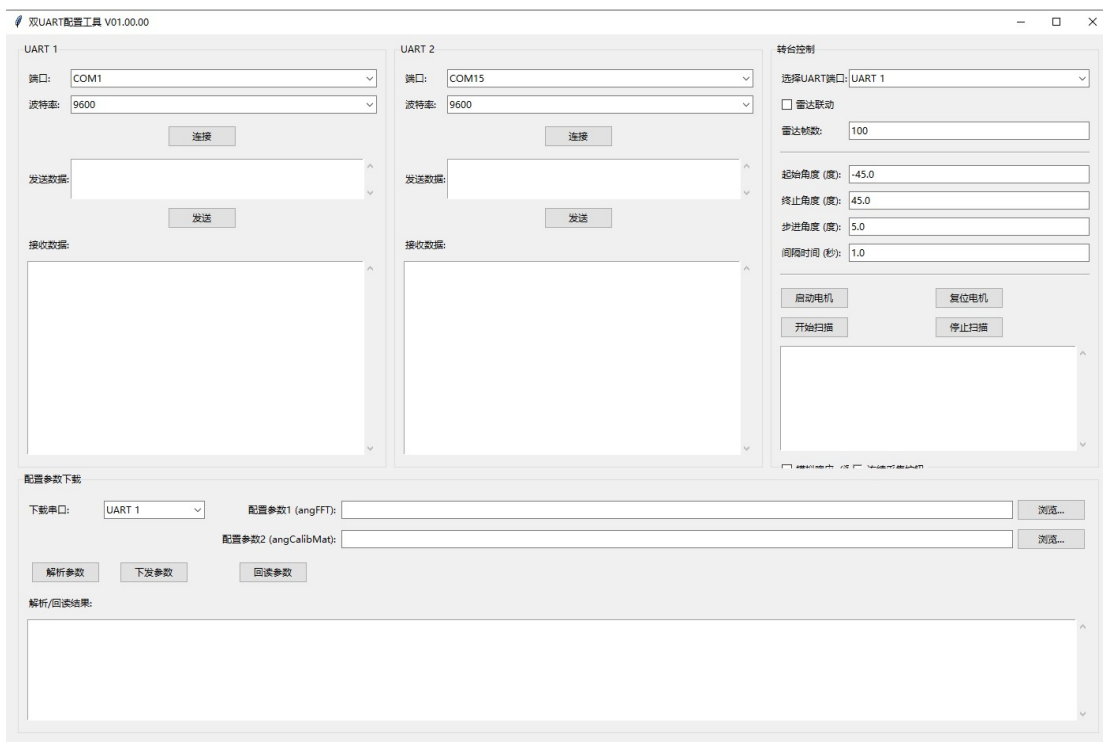


图 19 配置工具界面

- 选择 UVC serial 串口，即雷达上电后出现的串口号，点击连接可以看到按钮变成断开，且信息窗口显示串口连接成功。然后在下载串口选择 UVC serial 对应的串口组（UART 1 或 UART 2）

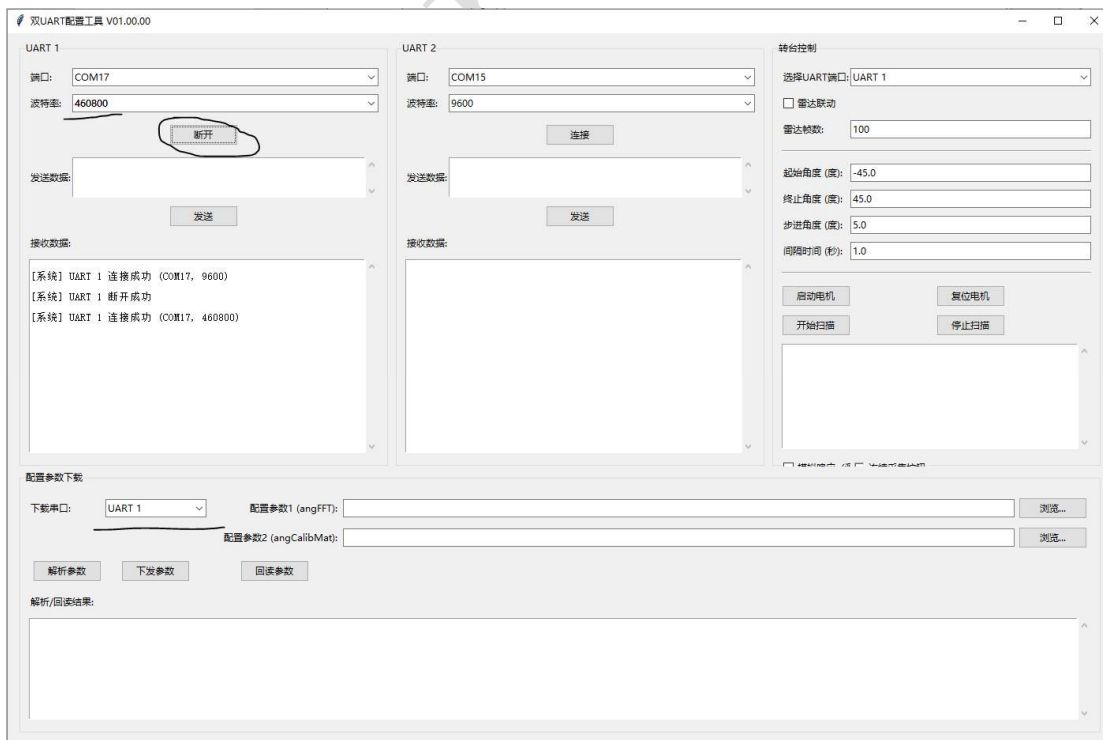


图 20 连接串口

## 5. 在配置参数中选择对应的.mat 文件



图 21 选择配置文件

## 6. 点击解析参数，然后选择下发参数，如果正常可以看到参数下发成功。

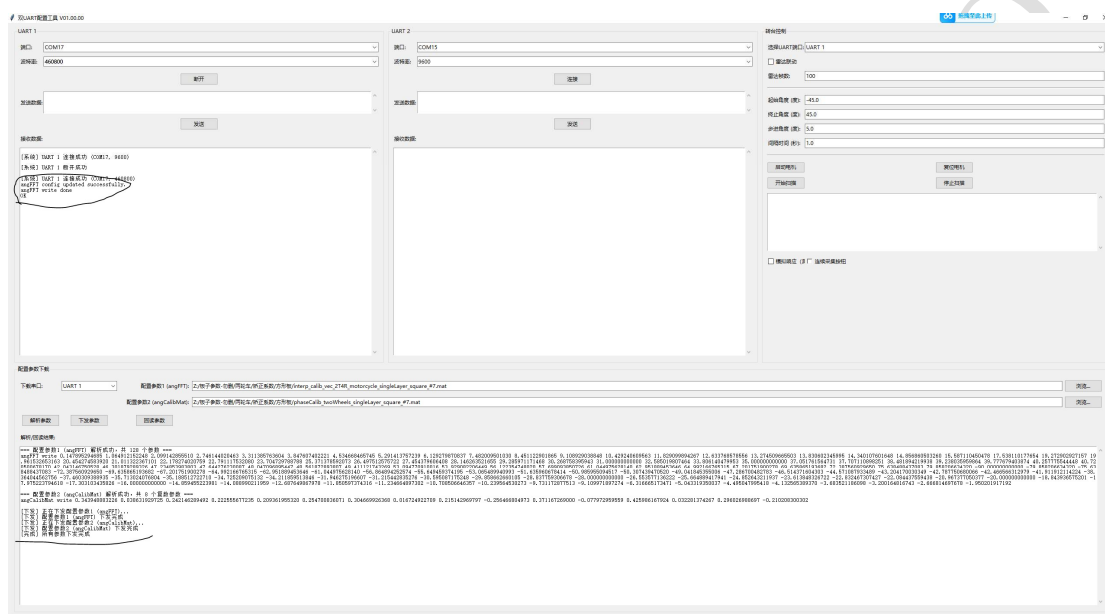


图 22 配置参数下载

## 7. 雷达断电后，重新上电，然后重新连接串口，回读配置参数，可以看到配置参数仍然正确，证明配置参数已经写入 Flash。

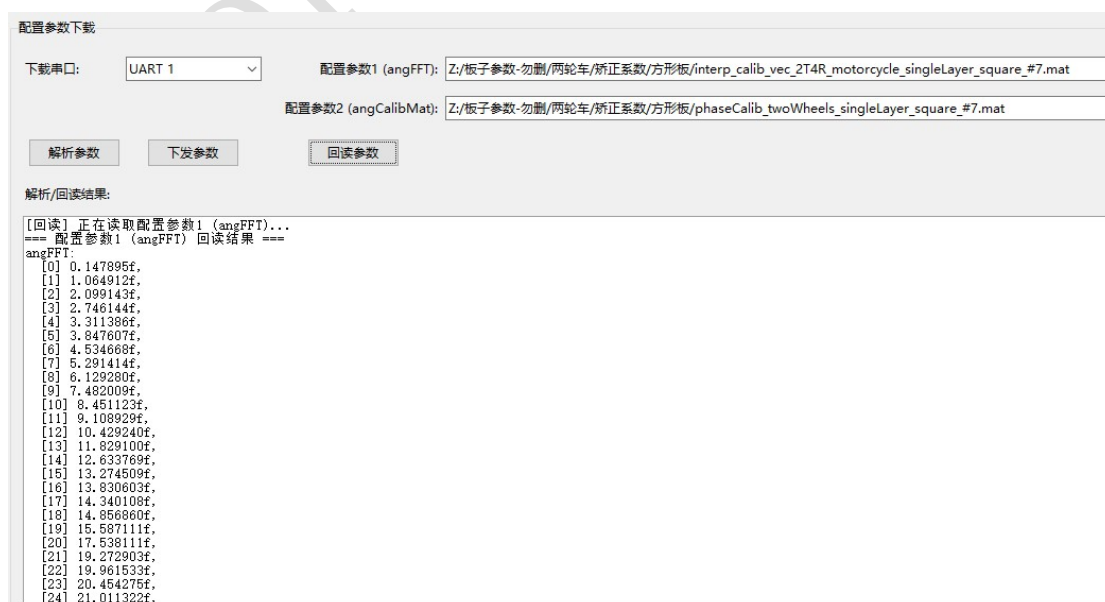




图 23 配置参数回读

而且重新升级后, 配置参数不丢失, 可以确认功能正确。



图 24 工具下载地址

## 五、OTA 升级操作

### 1 首次 OTA 下载

- 在代码编译时，在文件 x2100\_Cheetah\_nor\_defconfig 的第一行需要选择 CONFIG\_SPL\_BIN\_FILE 对应的启动 SPL 文件。

```
CONFIG_SPL_BIN_FILE=xburst2/soc-x2000/spl/x21001-spl-sfc-nor-1200M-500M.bin
```

图 25 原 SPL 文件，不支持 OTA

```
CONFIG_SPL_BIN_FILE=xburst2/soc-x2000/spl/x21001-spl-sfc-nor-ota-1200M-500M.bin
```

图 26 新的 SPL 文件，支持 OTA

注意：SPL 文件是固定的启动文件，不参与 APP 的更新

- 针对 APP 的编译指令与之前一样，编译完成后不再获取 rtos-with-spl.bin，而是获取 zero.bin 文件

```
senardmirco@DESKTOP-NRMJETT:~/MotorCycle_Ingenic/freertos$ cp -fv zero.bin /mnt/d/'zero.bin' -> '/mnt/d/zero.bin'
```

图 27 获取 APP 的固件

- 启动下载上位机并进入配置选项，选择 x2100x\_cheetah\_sfc\_nor\_ota\_freertos.cfg 配置文件

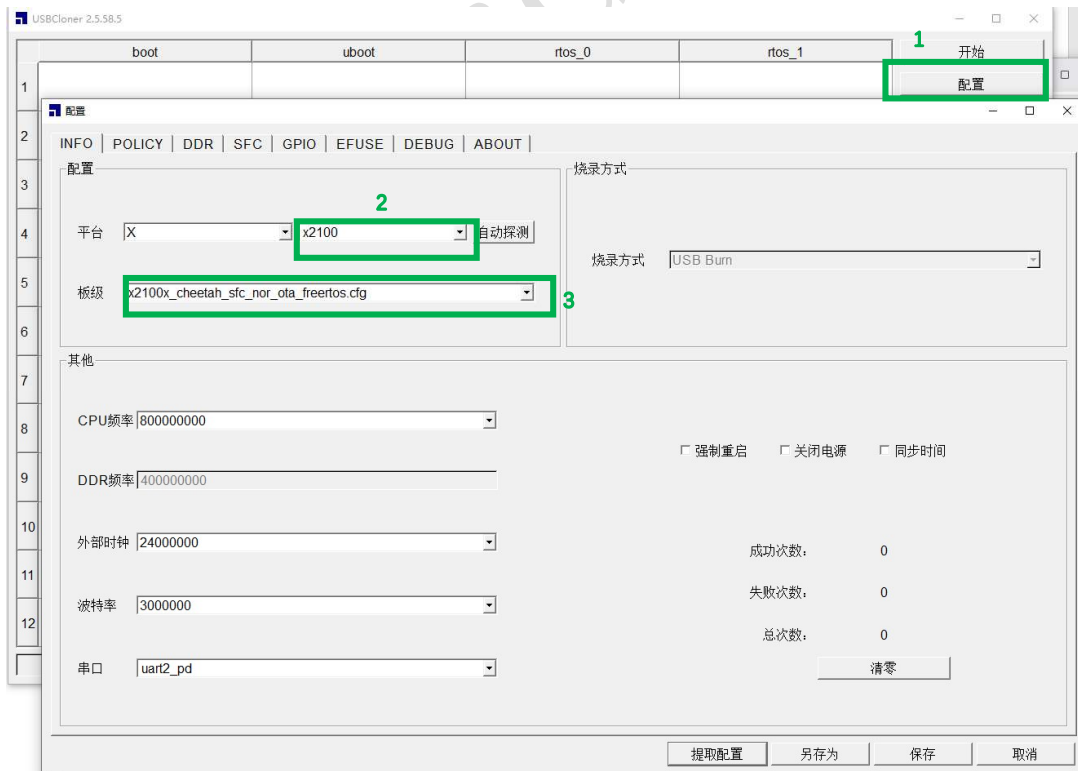


图 28 选择配置文件

#### 4. 选择下载的 uboot 文件和 APP 文件

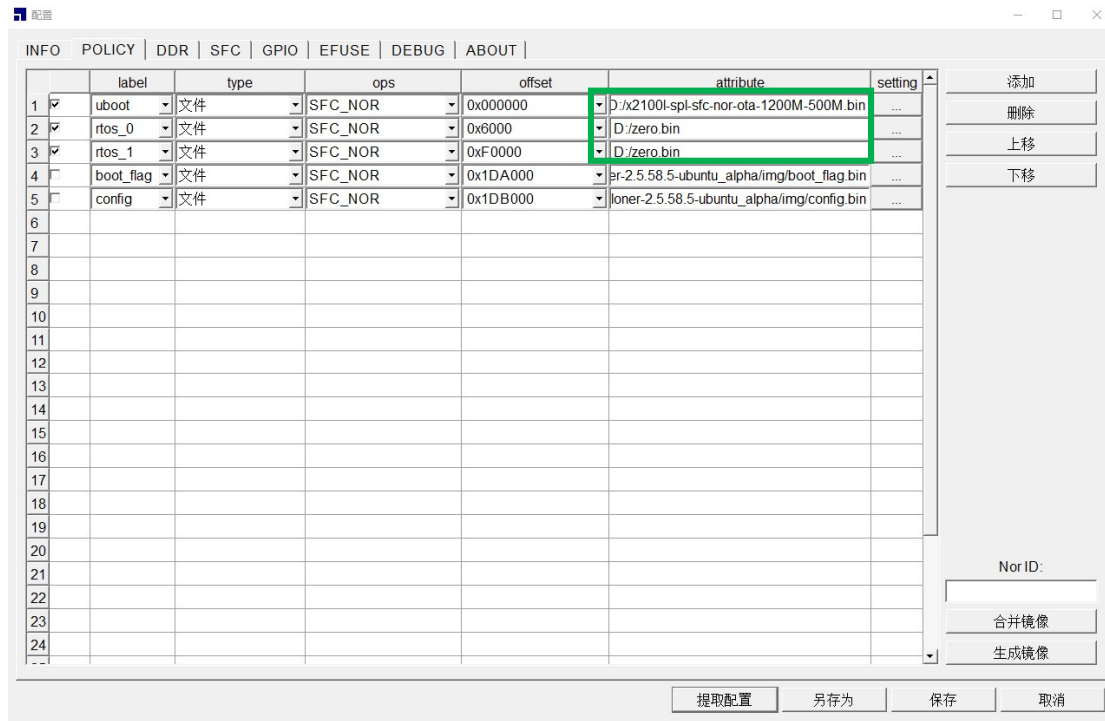


图 29 选择下载文件

#### 5. 首次下载时, 选择 Chip Erase 保证新的分区生效

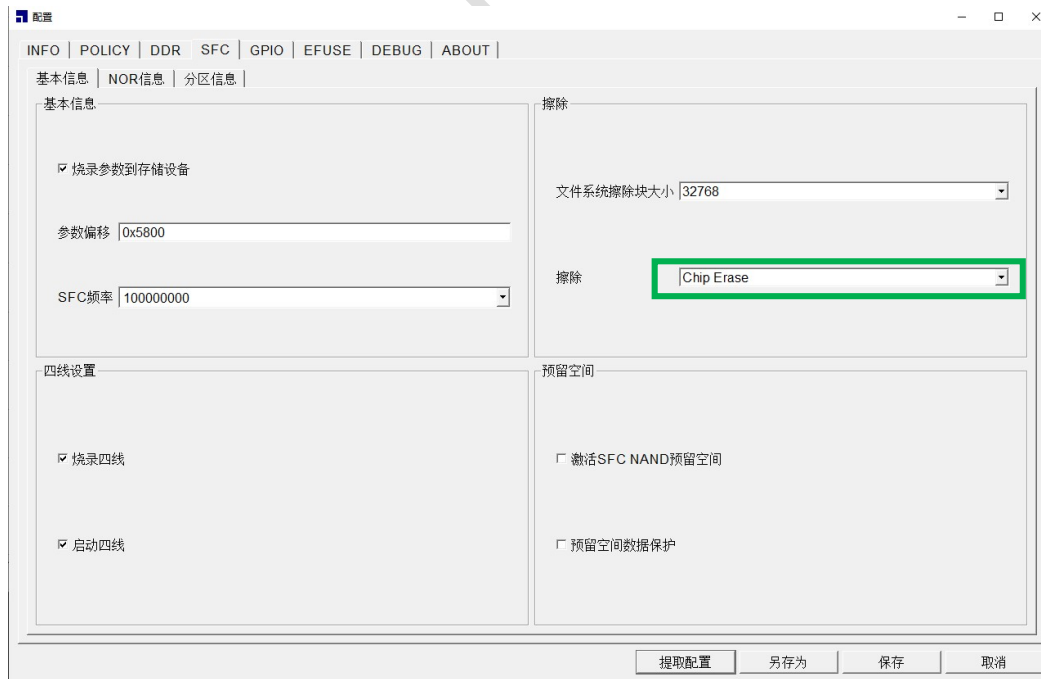
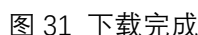


图 30 选择 Chip Erase

#### 6. 执行下载操作



The screenshot shows a Windows desktop environment. At the top, there are taskbar icons for various applications like Chrome, VS Code, and File Explorer. A search bar is visible in the upper left corner.

The main focus is a terminal window titled "6. COM16 USB Serial Port (COM16)". It contains the following output:

```
[E] 481397| UART receive task initialized successfully with semaphore protection
[E] 481244| wmt dual speed [N/A]pin 0[O/N]pin 10[O/N]pin 11[O/N]pin 12
[E] 484542| cfg 178015:8000 speeds: high full
[E] 485521| &serial ready
[E] 470105| ===== [RADAR_CONFIG] Loading(1.473183) usb serial: receive thread started
Configuration=====
[E] 472490| [RADAR_usb_serial]: read error (-87), wait and retry
CONFIG Step 1: Allocating heap buffer (Max: 40960 bytes)...
[E] 472545| UART receive task started (optimized version)
[E] 478880| [RADAR_CONFIG] Valid Config Lines Parsed: 84
[E] 480484| [RADAR_CONFIG] Infer Using Factory default Configuration...
[E] 481743| [RADAR_CONFIG] Step 2: Applying config to Radar hardware...
[E] 513177| usb serial: read error (-87), wait and retry
[E] 612269| USB new device is high-speed
[E] 612101| USB device new address 1
[E] 677273| usb serial: read error (-87), wait ahigh-speed config #1: CDC ACM config
[E] 678268| usb serial connect!!
[E] 680236| serial_connect_callback i
rd error:
[E] 680465| usb serial: adIDRate= 152500, bCharFormat 0, bParityType 0, bDataBits 8
[E] 680466| [RADAR_CONFIG] ==>=152500 Config ok! end! see
[E] 480779| ===== [RADAR_CONFIG] Process Complete =====

(E) 489780| Uzc_bus: -1 not find
(E) 500408| chetaht: failed to request Uzc bus: 1
(E) 502308| camera found sensor_chetaht (2048128) 524288
(E) 513380| CPU 1 is init
(E) 725881|

##### Frame 1 #####
(E) 728121| !!! cs\获取ADC总时间: 2 毫秒
(E) 731405|
温度: 47.45 °C
(E) 732253|

*** freect 1 *** main thread 总耗时: 45.44 毫秒
(E) 733783|

##### Frame 2 #####
(E) 738450| !!! cs\获取ADC总时间: 4 毫秒
(E) 834003|
温度: 46.10 °C
(E) 834193|

*** freect 2 *** main thread 总耗时: 50.46 毫秒
(E) 836193|

##### Frame 3 #####
(E) 839700| !!! cs\获取ADC总时间: 1 毫秒
(E) 850138|
温度: 44.70 °C
(E) 850922|

*** freect 3 *** main thread 总耗时: 47.87 毫秒
(E) 860784|

##### Frame 4 #####
(E) 869410| !!! cs\获取ADC总时间: 1 毫秒
(E) 870178|
温度: 43.37 °C
(E) 870200|

*** freect 4 *** main thread 总耗时: 47.53 毫秒
(E) 873000|

##### Frame 5 #####
(E) 948430| !!! cs\获取ADC总时间: 1 毫秒
(E) 959247|
温度: 43.11 °C
(E) 960390|

*** freect 5 *** main thread 总耗时: 47.59 毫秒
(E) 960733|

##### Frame 6 #####
(E) 960768| !!! cs\获取ADC总时间: 1 毫秒
(E) 963597|
温度: 42.21 °C
```

图 32 上电后雷达正常运行

以上为首次下载支持 OTA 程序的流程

## 2 OTA 升级流程

1. 打开上位机 MotorCycle\_Tools release[2.3.5], 注意只有 2.3.5 以后的版本, 才支持 OTA 操作。选择 OTA 选项卡

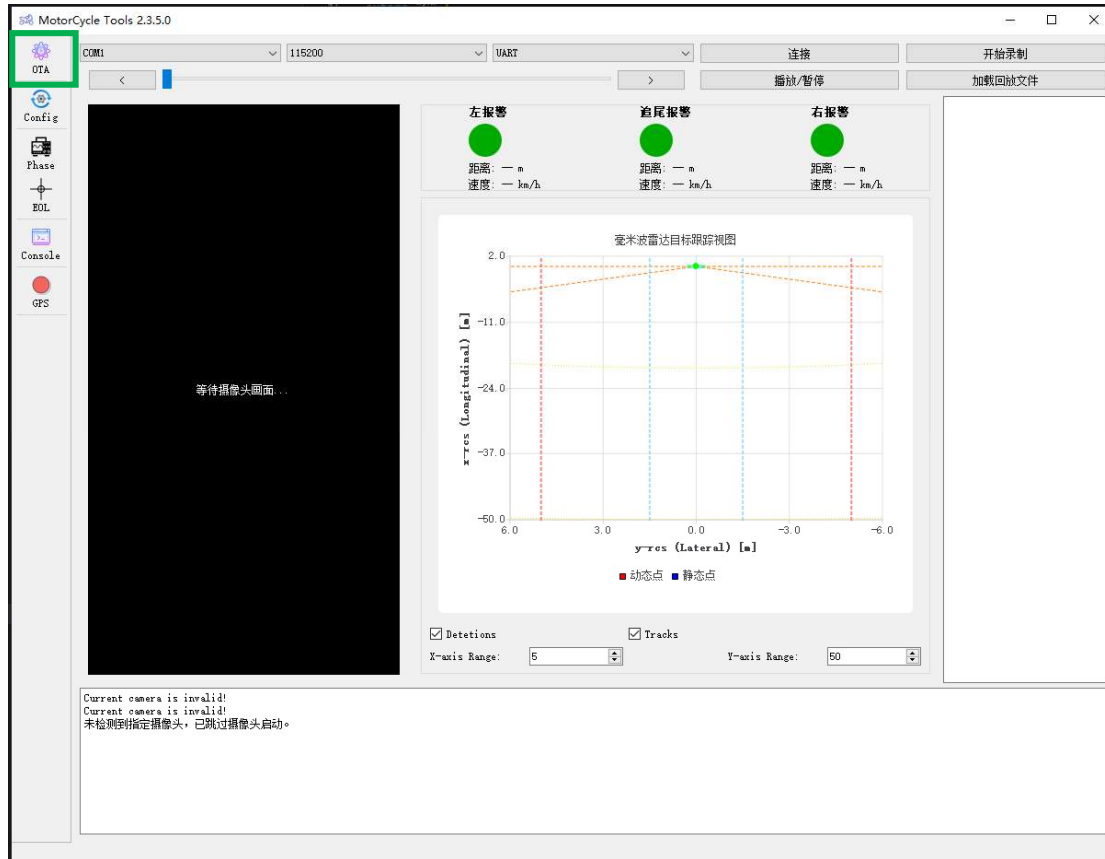


图 33 2.3.5 上位机

- 2.

## 六、ADC 数据采集

1. 雷达配置为 gadget serial 通讯, 参考第三章
2. 雷达上电后打开上位机, 可以看到输出的轨迹信息

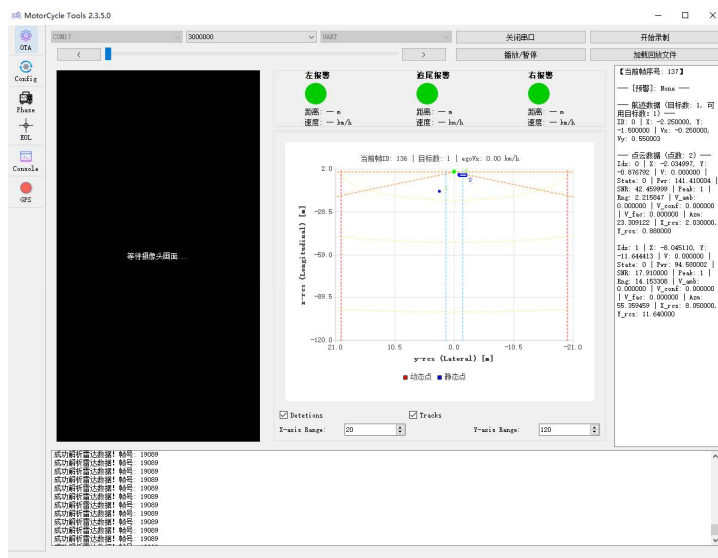


图 34 观察到轨迹信息

3. 关闭串口, 打开 Console 界面, 在 Console 界面下连接 gadget serial 端口。选择端口后配置帧数为 0, 选择发送

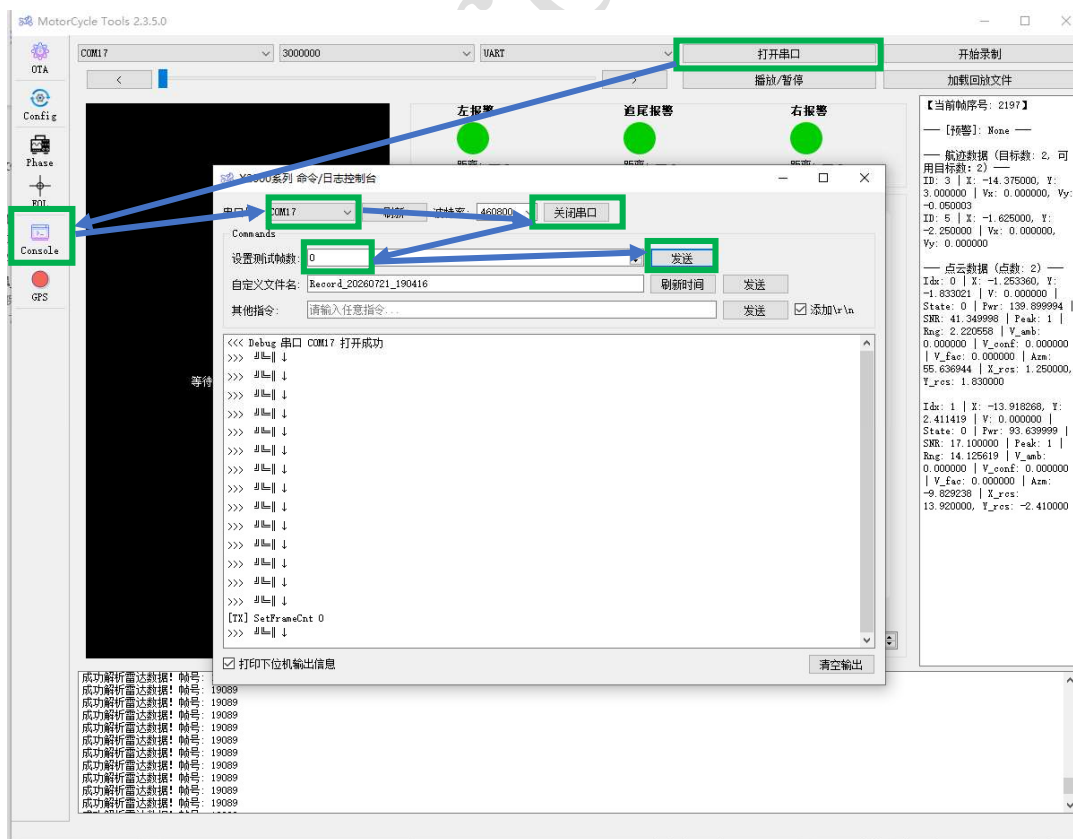


图 35 操作暂停雷达工作

- 在其他指令中输入 setRawDataFlg 1 并点击发送。

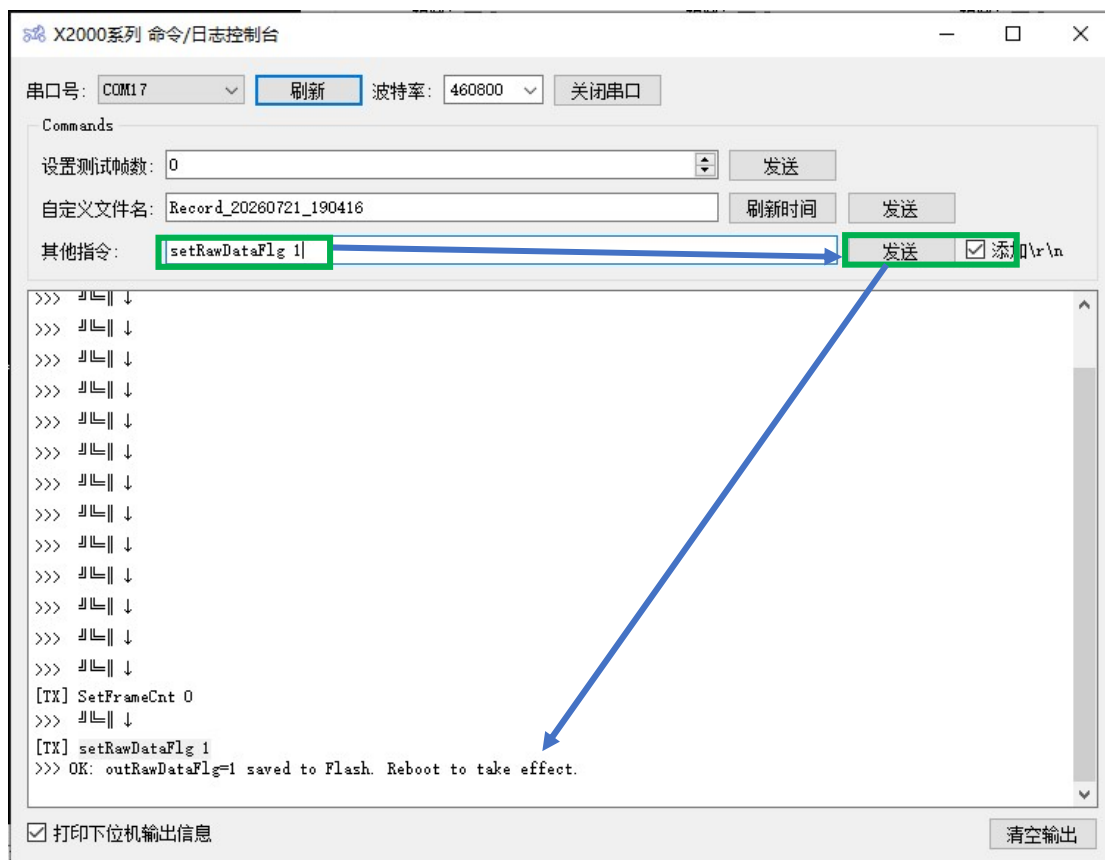


图 36 发送指令，切换到输出 ADC 模式

- 雷达断电在上电后，通过 gadget serial 默认输出的就是 ADC 数据了。

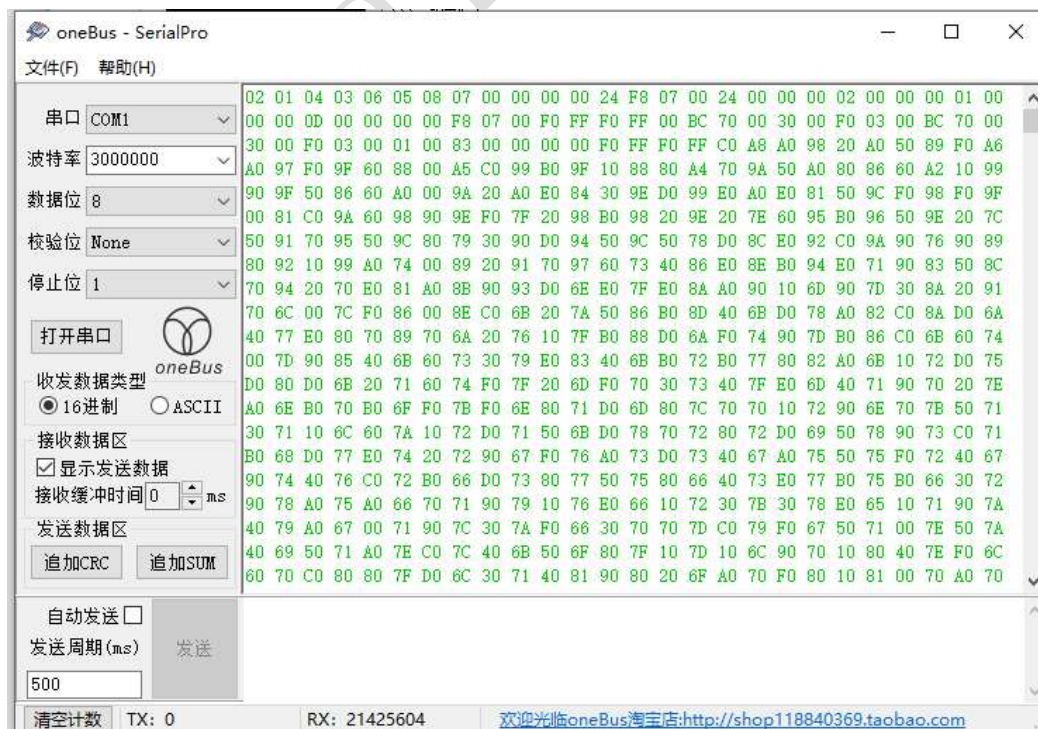


图 37 串口调试助手观察输出的 ADC 数据

6. 通过 ADC 脚本解析结果，如下图所示：

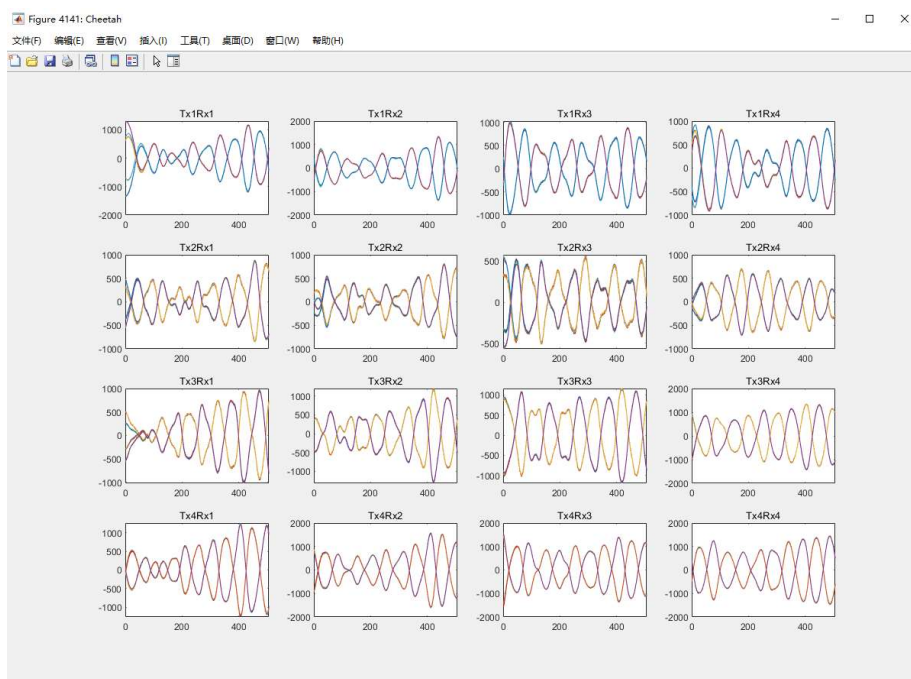


图 38 ADC 通道解析结果

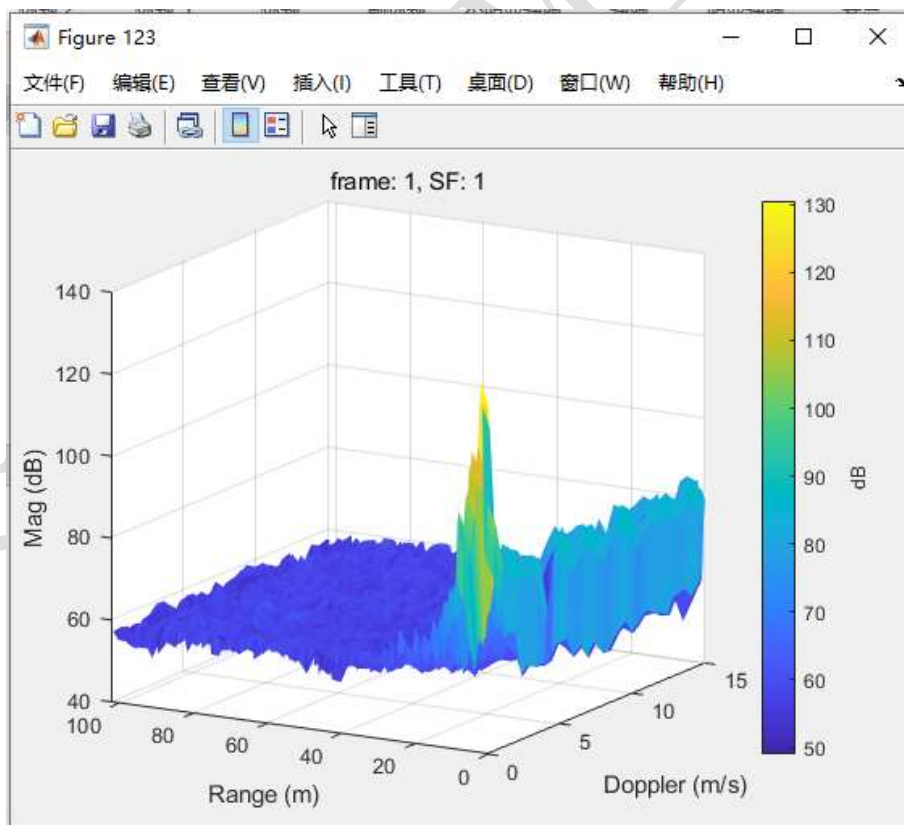


图 39 ADC 绘制 RD 图

7. 通过上位机存储 ADC 数据，上位机版本必须大于 2.3.6.0

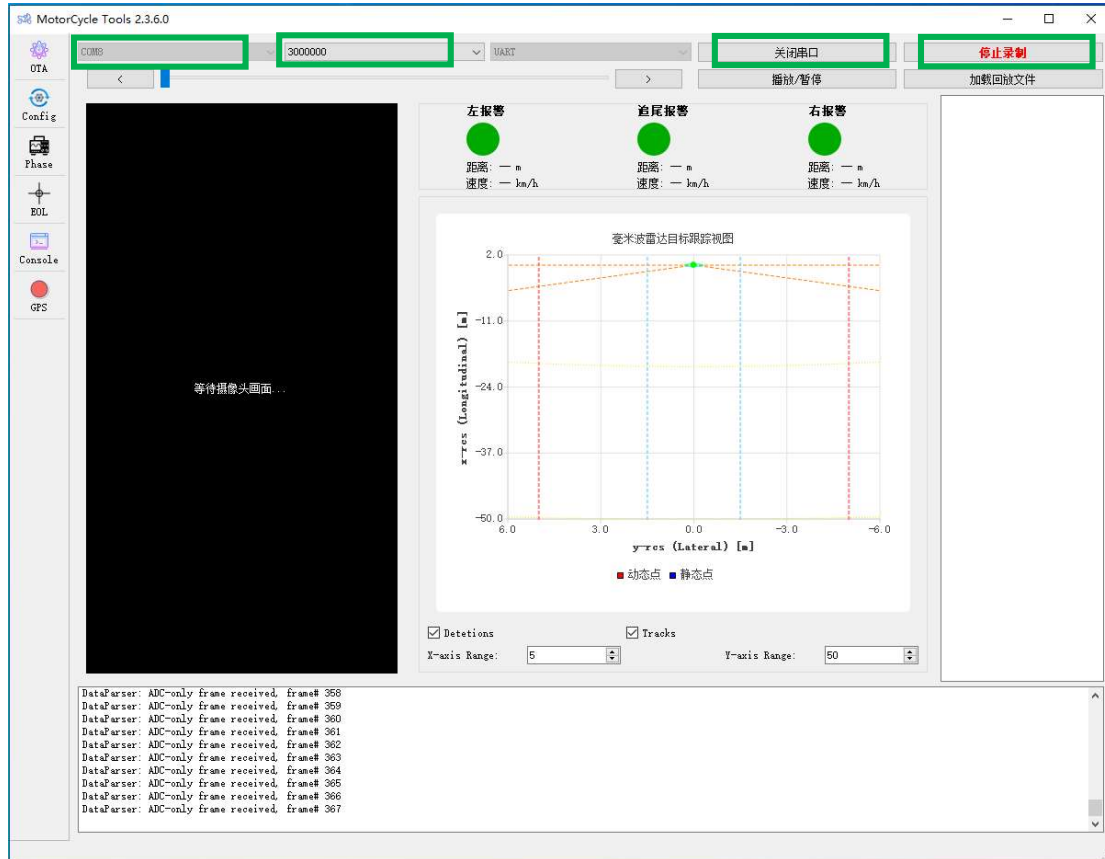


图 40 上位机存储 ADC 数据

## 8. 可以在 data 文件夹下找到 ADC 数据

新加卷 (E:) > x2000 > MotorCycle\_Tools release[2.3.6] > MotorCycle\_Tools release[2.3.6] > data > Record\_2026\_07\_22

| 名称                             | 修改日期            | 类型     | 大小         |
|--------------------------------|-----------------|--------|------------|
| Record_20260722_135448_adc.dat | 2026/7/22 13:55 | DAT 文件 | 184,633 KB |

图 41 ADC 数据结果位置

## 9. 对 ADC 数据进行解析, 结果如下

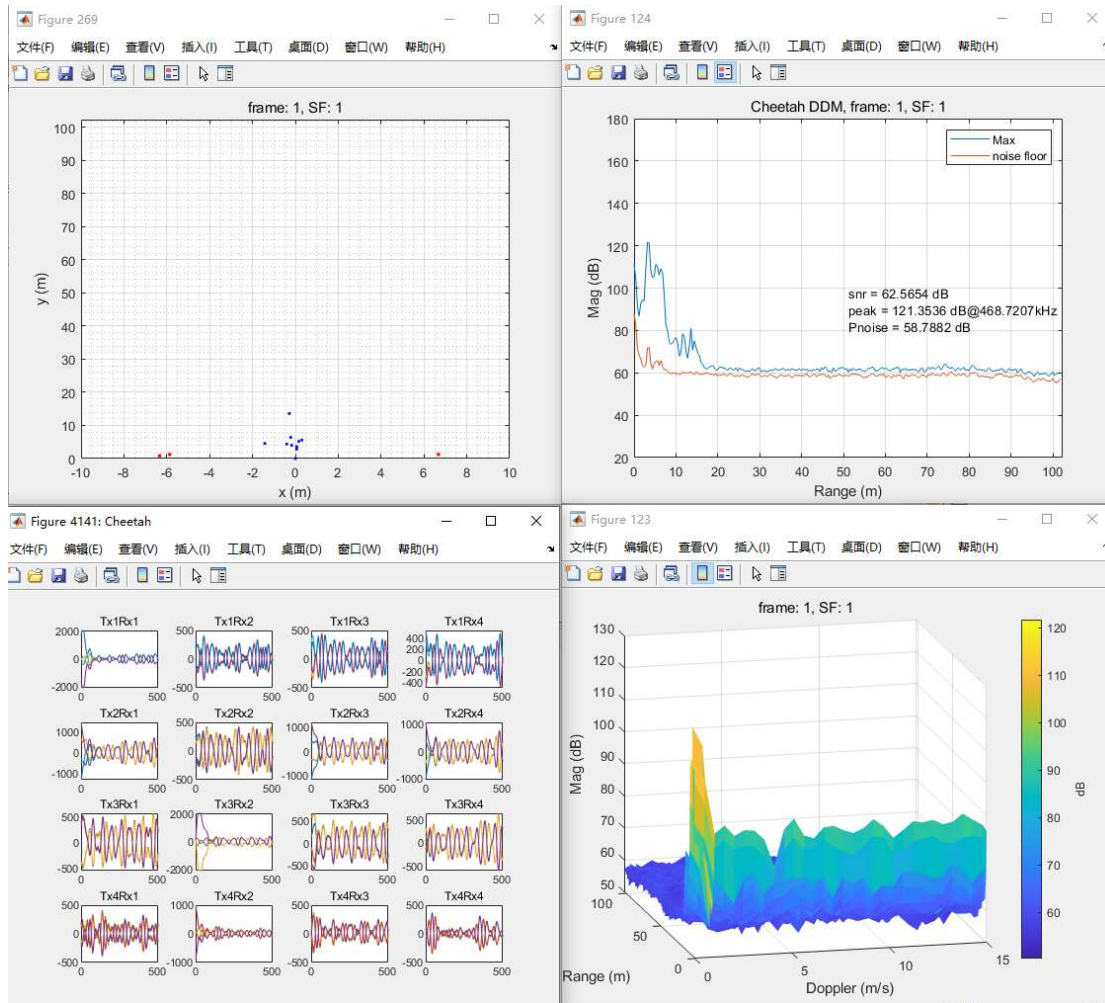
```
命令窗口
=== 格式参数配置 ===
num_chirp      = 128
num_adc (输出) = 506
num_adc_src (源文件) = 506
N_RX_PER_MMIC = 4
CHIRP_HEADER_BYTES = 32
PACKET_HEADER_BYTES = 76
PACKET_PAD_ZERO = 16
payload_length = 522240 (0x7F800)
packet_align_bytes = 4
每帧总大小 (输出) = 522336 bytes
源文件每 chirp bytes = 4080 (纯 ADC, 无 header)
源文件每帧 bytes = 522276 (36 + 522240)

找到 1 个源文件, 开始转换...

[1/1] 处理: Record_20260722_135448_adc.dat
源文件格式: .dat
输出文件: Record_20260722_135448_adc_Zu3Format.dat
源文件大小: 189063912 bytes (180.31 MB)
源文件前 32 字节 (hex): 02 01 04 03 06 05 08 07 00 00 00 00 24 F8 07 00 24 00 00 00 06 00 00 00 01 00 00 00 0D 00 00 00
检测到 362 帧 (每帧 522276 字节, header=36B, ADC=522240B)
源文件每 chirp: 4080 bytes = 32B chirp header + 4 RX × 506 samples × 2 bytes
输出每 chirp: 32 bytes header + 4048 bytes ADC (4 RX × 506 samples × 2 bytes)
=> 输出: Record_20260722_135448_adc_Zu3Format.dat (180.35 MB)

转换完成!
提示: 在 RadarParser_Zu3_Cheetah_4D_2T4R_x2000convZu3.m 中
filename = 'D:\yufeng\01_Project\MotorCycle_Ingenic\06_Verfication\Record_2026_07_22\Record_20260722_135448_adc_Zu3Format.dat'
```

图 42 ADC 格式转换结果



ADC 解析结果

## 七、常用指令集

| 指令名称                  | 指令行             | 参数                                   | 备注                                     |
|-----------------------|-----------------|--------------------------------------|----------------------------------------|
| 设置当前执行帧数              | SetFrameCnt     | -1 (持续运行)<br>> 0 (运行的帧数)<br>0 (立即停止) | 配置当前上电周期内的运行帧数, 默认为-1, 断电复位            |
| 设置上电后运行帧数             | SetMaxFrameCnt  | -1 (持续运行)<br>> 0 (运行的帧数)<br>0 (立即停止) | 配置上电运行帧数, 断电保存                         |
| SD 卡文件名配置             | SetDumpFileName | 字符串                                  | 写入 SD 卡存储的文件名                          |
| 读寄存器                  | readreg         | 寄存器地址                                | 适配 Cheetah 的 Radar Tool 指令             |
| 写寄存器                  | writereg        | 寄存器地址+寄存器值                           | 适配 Cheetah 的 Radar Tool 指令             |
| 延迟                    | delay           | 延迟时间                                 | 适配 Cheetah 的 Radar Tool 指令             |
| 复位寄存器                 | ResetRams       | 复位寄存器                                | 适配 Cheetah 的 Radar Tool 指令             |
| 获取板端版本号               | getBoardVersion | Null                                 | 适配 Cheetah 的 Radar Tool 指令             |
| 获取温度                  | getTemperature  | Null                                 | 获取 Cheetah 当前的芯片温度                     |
| 设置当前雷达转台角度            | setAngle        | 666 (无效值)<br>-90°~90°                | 配合转台上位机及 Matlab 校准脚本实现相位校准             |
| 配置初相校准补偿              | angCalibMat     | read 读取结果<br>write 写入结果              | 算法计算后的参数可以被写入 Flash, 掉电不丢失, 重新上电后生效    |
| 配置 FFT 补偿             | angFFT          | read 读取结果<br>write 写入结果              | 算法计算后的参数可以被写入 Flash, 掉电不丢失, 重新上电后生效    |
| EOL 校准操作              | uds             | 校准操作参数                               | 适用于客户下线校准                              |
| 配置 gadget serial 输出信息 | setRawDataFlg   | 0: 输出点云+轨迹<br>1: 输出 ADC              | 仅 gadget serial 输出信息时生效。物理 UART 输出时不生效 |
| OTA 升级启动命令            | otaUpgrade      | Null                                 | 启动 OTA 升级                              |